



郑州大学
Zhengzhou University

第四单元 哈希函数与消息认证码

郑州大学网络空间安全学院

王志华

王志华



第4单元 哈希函数与消息认证码

提纲

CONTENTS

4.1 哈希函数概述

4.2 MD算法

4.3 SHA算法

4.4 MAC

4.5 SM3

4.6 其他哈希函数

2022/4/24



4.3.1 SHA算法概述

●SHA-1 — “secure hash algorithm”

- 1995年美国国家安全局设计，并由美国国家标准技术研究所（NIST）发布为**联邦数据处理标准**（FIPS）
- 160比特输出
- 2005年，密码分析人员发现了对SHA-1的有效攻击方法，这表明该算法可能不够安全，不能继续使用
- 自2010年以来，许多组织建议用SHA-2或SHA-3来替换SHA-1

2022/4/24



1、SHA-1算法的历史

- SHA是基于MD4算法，其结构与MD4非常类似。
- SHA-0是SHA的早期版本，SHA-0被公布后，NIST很快就发现了它的缺陷，修改后的版本称为SHA-1，简称为SHA。
 - SHA于1992年1月31日在联邦记录中公布。
 - SHA于1993年作为联邦信息处理标准（FIPS PUB 180）公布。
 - 1994年7月11日作了一个小的修改。
 - 1995年4月17日公布了修改后的版本，称为SHA-1。
 - 1995年5月11日起采纳为国家标准(SHS)。

王卫华

4.3.2 SHA-1算法

- ❖ **RFC 3174**也给出了**SHA-1**，它基本上是复制**FIPS 180-1**的内容，但增加了**C**代码实现。
- ❖ **SHA-1**算法的输入是长度小于 2^{64} 的任意消息**x**，输出**160**位的散列值。



2022/4/24



1、SHA-1算法概述

- 算法的**输入**为小于 2^{64} 比特长的任意消息，分为**512比特长的分组**；
- 算法的**输出**为**160比特长的消息摘要**；
- 算法的**链接变量**的长度为**160比特**。

王卫华



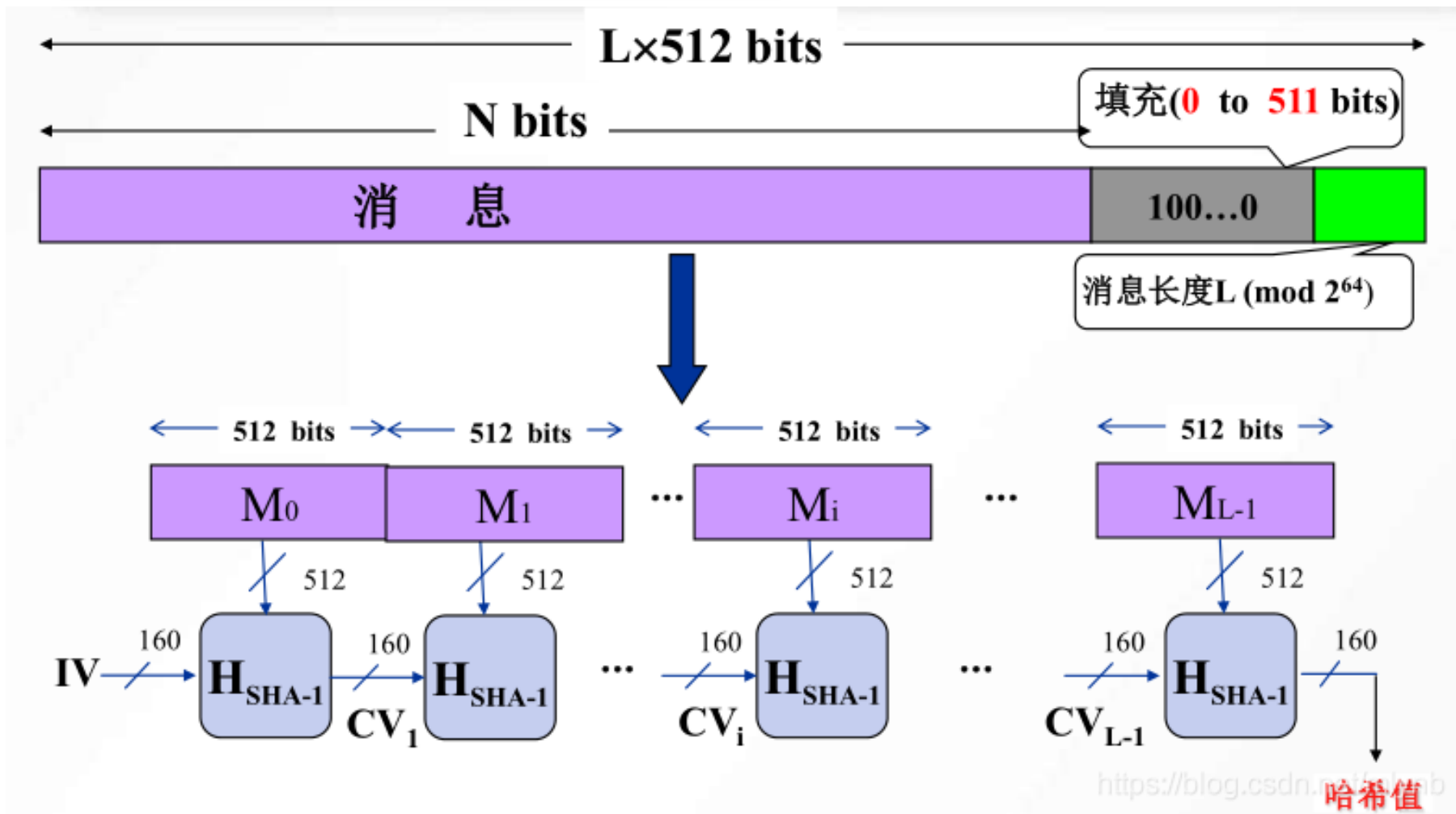
2、SHA-1算法描述：流程

- SHA-1算法的流程：
 - 填充消息
 - 初始化变量
 - 扩展消息
 - 进行主循环
 - 输出

王华



2、SHA-1算法描述：流程



2022/4/24



2、SHA-1算法描述：第一步

● 处理过程：1. 对消息填充

① 对消息填充与MD5的步骤（1）完全相同：

□ 对消息填充，使得其比特长 $\pmod{512}$ 为448，即填充后消息的长度为512的某一倍数减64，留出的64比特备第2步使用。

- 步骤1是必需的，即使消息长度已满足要求，仍需填充。
- 例如，消息长为448比特，则需填充512比特，使其长度变为960，因此填充的比特数大于等于1而小于等于512。
- 填充方式是固定的：第1位为1，其后各位皆为0

2022/4/24



2、SHA-1算法描述：第二步

- 处理过程：**2. 附加消息的长度**
 - ② 附加消息的长度**与MD5的步骤(2)类似**，不同之处在于以**大端方式**表示填充前消息的长度。
 - 将步骤(1)留出的64比特当作64比特长的无符号整数。

2022/4/24



2、SHA-1算法描述：前两步

- 处理过程有以下几步：

❖ 步骤1与步骤2一起称为消息的预处理

经预处理后，原消息长度变为**512**的倍数。设原消息 x 经预处理后变为消息 $M=M_0M_1\dots M_{N-1}$ ，其中 M_i ($i=0,1,\dots,N-1$) 是**32**比特。在后面的步骤中，将对**512**比特的分组进行处理。

2024



2、SHA-1算法描述：第三步

- 处理过程： 3. 对MD缓冲区初始化

③ 对MD缓冲区初始化算法使用160比特长的缓冲区存储中间结果和最终哈希值，缓冲区可表示为五个32比特长的寄存器（A，B，C，D，E），每个寄存器都以大端方式存储数据：

➤ 初始值分别为：A=67452301，B=EFCDA89，
C=98BADCFB，D=10325476，E=C3D2E1F0

2022/4/24



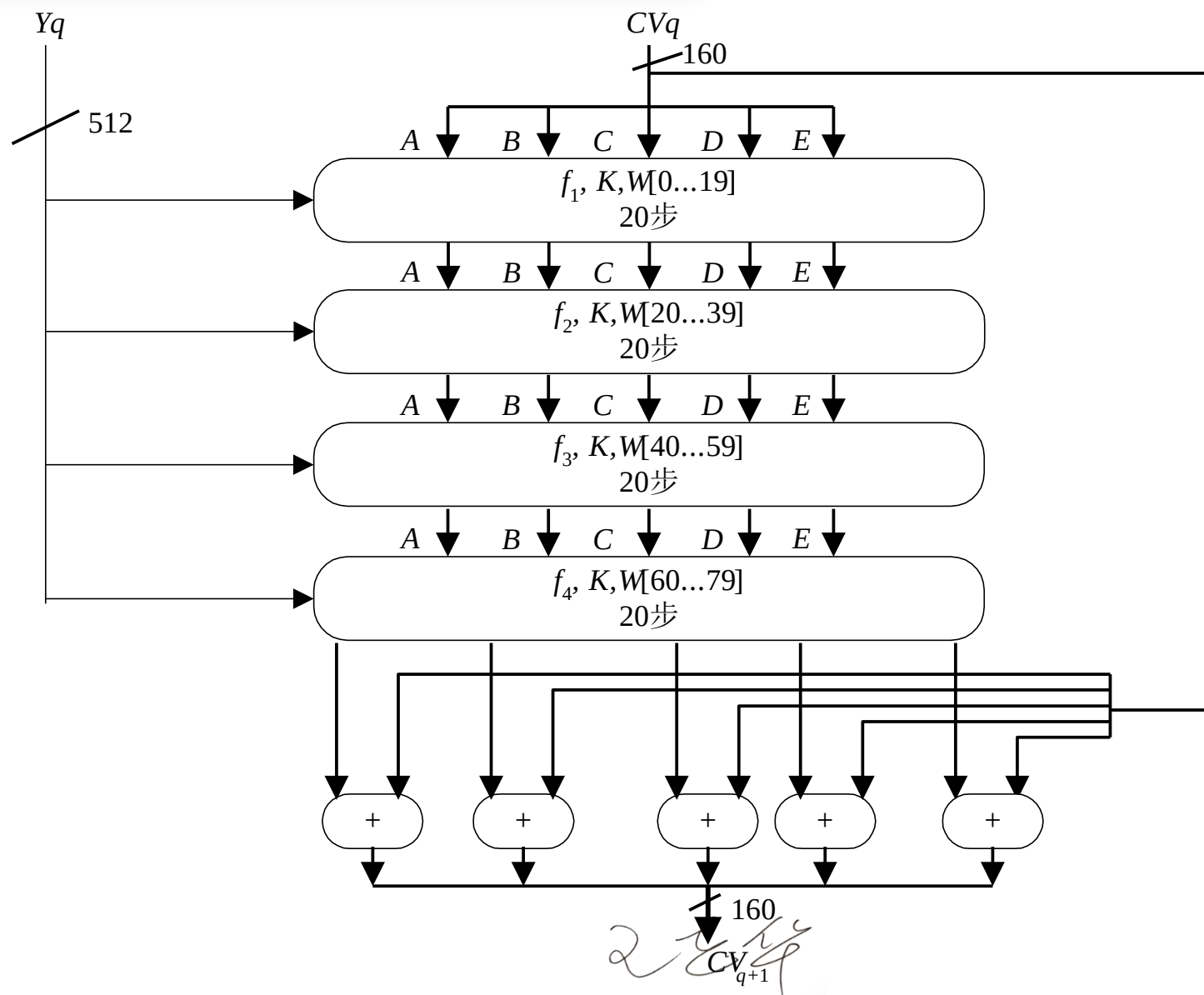
2、SHA-1算法描述：第四步

- 处理过程：4. 以分组为单位对消息进行处理
 - ④ 每一分组都经一压缩函数处理，压缩函数由4轮处理过程构成，每一轮又由20步迭代组成。

2024



2、SHA-1算法描述：第四步





2、SHA-1算法描述：第四步

- 处理过程：**4. 以分组为单位对消息进行处理**
 - 与MD的4轮**处理过程结构一样**，但所用的**基本逻辑函数不同**，分别表示为 f_1, f_2, f_3, f_4 。
 - **每轮的输入**为当前处理的消息分组 Y_q 和缓冲区的当前值A、B、C、D、E。
 - **每轮的输出**仍放在缓冲区以替代A、B、C、D、E的旧值。

王卫华



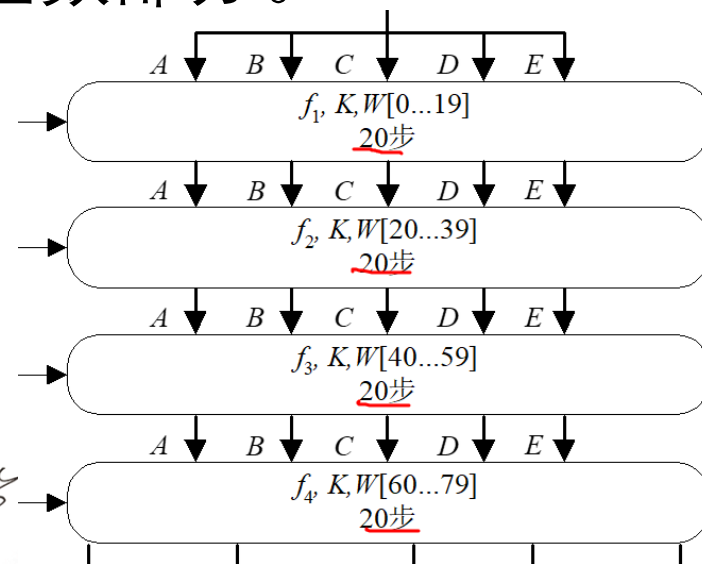
2、SHA-1算法描述：第四步

● 处理过程：4. 以分组为单位对消息进行处理

➤ 每轮处理过程还需加上一个加法常量 K_t ，其中 $0 \leq t \leq 79$ 表示迭代的步数。

➤ 80个加法常量 K_t 中实际上只有4个不同取值，如表6-7所示，其中 $\lfloor x \rfloor$ 为x的整数部分。

➤ K_t 类似于MD5中的常数表T[i]。



2022/4/24



2、SHA-1算法描述：第四步

表6-7 SHA的加法常量

迭代步数 t	常量 K_t (16 进制)	K_t (10 进制)
$0 \leq t \leq 19$	5A827999	$\lfloor 2^{30} \times \sqrt{2} \rfloor$
$20 \leq t \leq 39$	6ED9EBA1	$\lfloor 2^{30} \times \sqrt{3} \rfloor$
$40 \leq t \leq 59$	8F1BBCDC	$\lfloor 2^{30} \times \sqrt{5} \rfloor$
$60 \leq t \leq 79$	CA62C1D6	$\lfloor 2^{30} \times \sqrt{10} \rfloor$

2024



2、SHA-1算法描述：第四步

- 处理过程：4. 以分组为单位对消息进行处理
 - 第4轮的输出（即第80步迭代的输出）再与第1轮的输入 CV_q 相加，以产生 CV_{q+1} ，
 - 其中加法是缓冲区中5个字中的每一个字与 CV_q 中相应的字模 2^{32} 相加。

2022/4/24



2、SHA-1算法描述：第四步

❖ 步骤4: 以512位的分组（16个字）为单位处理消息

- SHA-1是迭代Hash函数，其压缩函数为：

$$H_{\text{SHA}} : \{0,1\}^{160+512} \rightarrow \{0,1\}^{160}.$$

- 步骤4是SHA-1算法的主循环，它以512比特作为分组，重复应用压缩函数 H_{SHA} ，从消息的第一个分组开始，依次对每个分组进行压缩，直至最后分组，然后输出消息 x 的Hash值。
- SHA-1循环次数等于消息中512比特分组的数目 L 。

2024



2、SHA-1算法描述：第五步

- 处理过程： **5.输出**

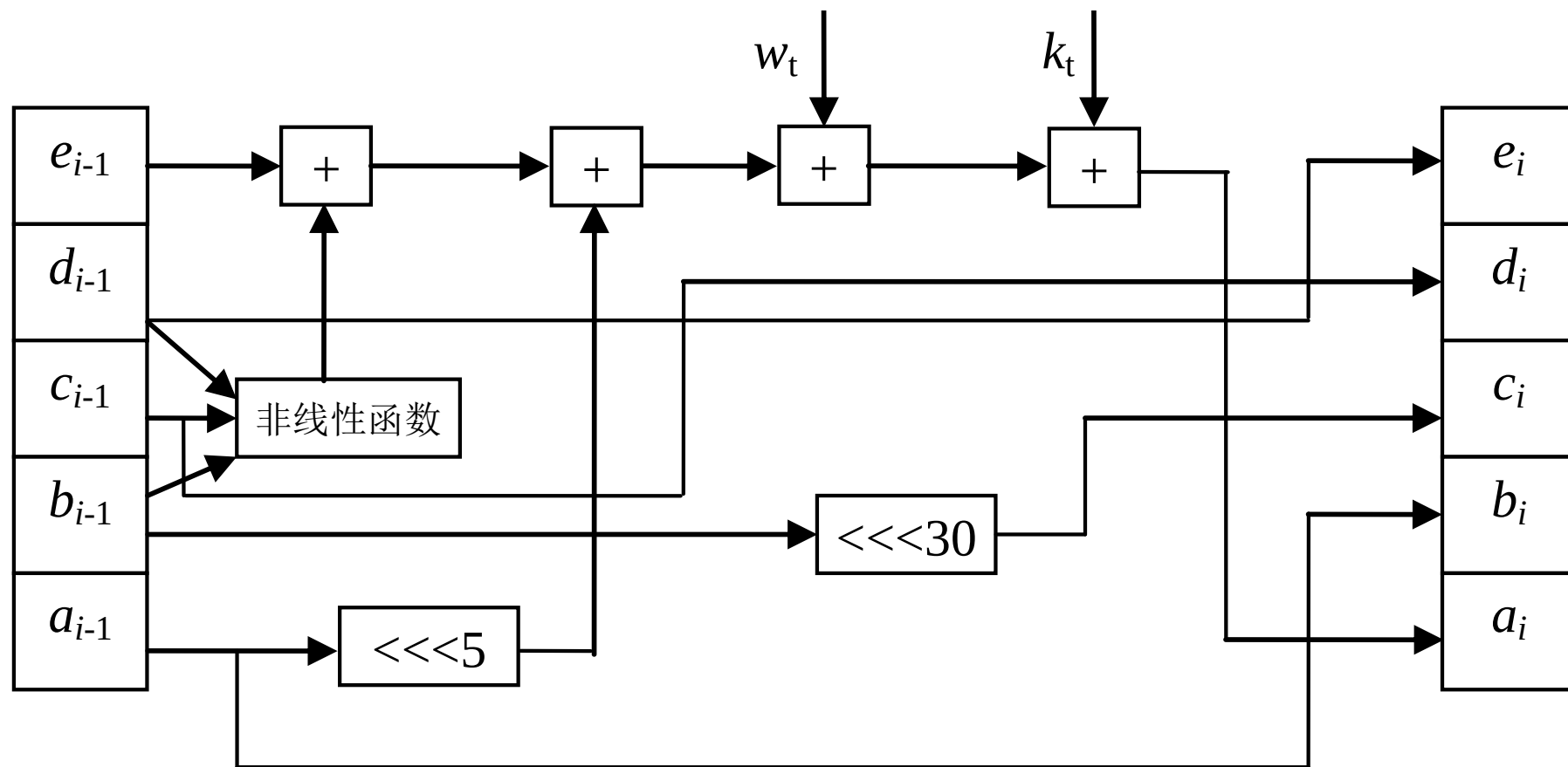
⑤ 输出消息的L个分组都被处理完后，最后一个分组的输出即为160比特的消息摘要。

王卫华



3、SHA-1的压缩函数

单个操作过程描述



2022/4/24



3、SHA-1的压缩函数

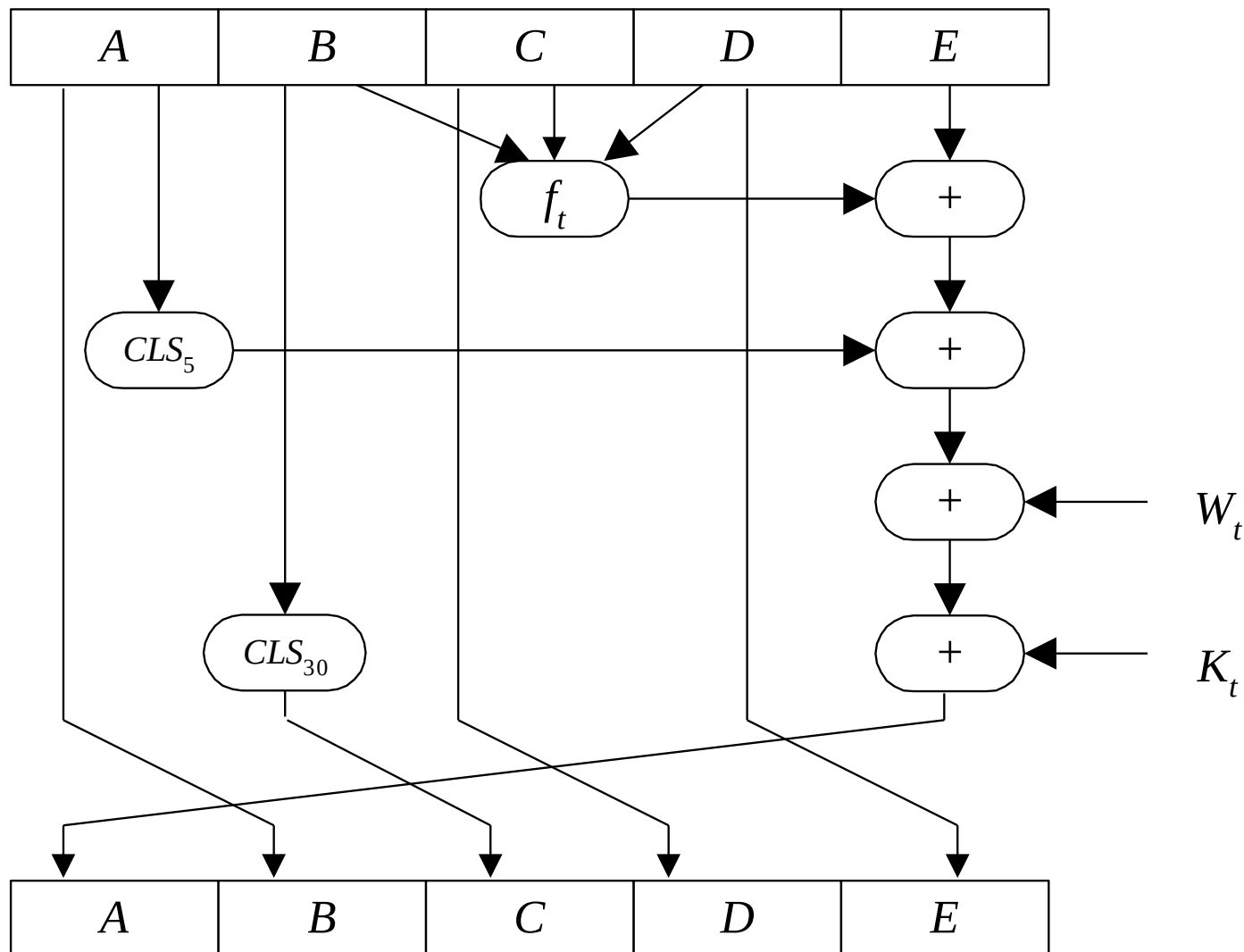
- SHA的压缩函数由**4轮**处理过程组成，每轮处理过程又由对缓冲区ABCDE的**20步**迭代运算组成，**每一步迭代运算的形式为：**

$$A, B, C, D, E \leftarrow (E + f_t(B, C, D) + CLS_5(A) + W_t + K_t), A, CLS_{30}(B), C, D$$

2024



3、SHA-1的压缩函数



2022/4/24



3、SHA-1的压缩函数

- 压缩函数 H_{SHA} 的四轮处理过程的算法结构相同，每一轮要对缓冲区**ABCDE**进行**20**次迭代，每次迭代的运算形式为

$$\text{TEMP} = (A \ll 5) + f_t(B, C, D) + E + X[k] + K_t$$

$$E = D$$

$$D = C$$

$$C = B \ll 30$$

$$B = A$$

$$A = \text{TEMP}$$

2024



3、SHA-1的压缩函数

$$A, B, C, D, E \leftarrow (E + f_t(B, C, D) + CLS_5(A) + W_t + K_t), A, CLS_{30}(B), C, D$$

- 压缩函数中的每一步：
 - 其中A, B, C, D, E为**缓冲区的五个字**,
 - t是**迭代的步数** ($0 \leq t \leq 79$) ,
 - $f_t(B, C, D)$ 是第t步迭代使用的**基本逻辑函数**,
 - CLS_s 为**左循环移s位**,
 - W_t 是由当前512比特长的**分组**导出的一个32比特长的字 (**类似于MD5中的T[i]**) ,
 - K_t 是**加法常量**,
 - +是模 2^{32} 加法。

2022/4/24



3、SHA-1的压缩函数：逻辑函数

- 基本逻辑函数的**输入**为3个32比特的字
- 基本逻辑函数的**输出**是一个32比特的字
- 基本逻辑函数的**运算**为逐比特逻辑运算，即输出的第 n 个比特是三个输入的相应比特的函数
- 基本逻辑函数**是非线性部件！实现了代换操作。**

王立华



3、SHA-1的压缩函数：逻辑函数

表6-8 SHA中基本逻辑函数的定义

迭代的步数	函数名	定义
$0 \leq t \leq 19$	$f_1 = f_t(B, C, D)$	$(B \wedge C) \vee (\bar{B} \wedge D)$
$20 \leq t \leq 39$	$f_2 = f_t(B, C, D)$	$B \oplus C \oplus D$
$40 \leq t \leq 59$	$f_3 = f_t(B, C, D)$	$(B \wedge C) \vee (B \wedge D) \vee (C \wedge D)$
$60 \leq t \leq 79$	$f_4 = f_t(B, C, D)$	$B \oplus C \oplus D$

其中 $\wedge, \vee, \neg, \oplus$ 分别是与、或、非、异或四个逻辑运算，函数的真值表如表6-9所示。

2022/4/24



3、SHA-1的压缩函数：逻辑函数

表6-9 SHA的基本逻辑函数的真值表

B	C	D	f_1	f_2	f_3	f_4
0	0	0	0	0	0	0
0	0	1	1	1	0	1
0	1	0	0	1	0	1
0	1	1	1	0	1	0
1	0	0	0	1	0	1
1	0	1	0	0	1	0
1	1	0	1	0	1	0
1	1	1	1	1	1	1

2022/4/24



3、SHA-1的压缩函数： W_t

- 下面说明如何由当前的输入分组（512比特长）导出32比特长的字 W_t ：
 - 前16个值（即 $W_0, W_1, \dots, W_{15}$ ）直接取为输入分组的16个相应的字
 - 其余值（即 $W_{16}, W_{17}, \dots, W_{79}$ ）取为：

$$W_t = CLS_1(W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3})$$

2022



3、SHA-1的压缩函数: W_t

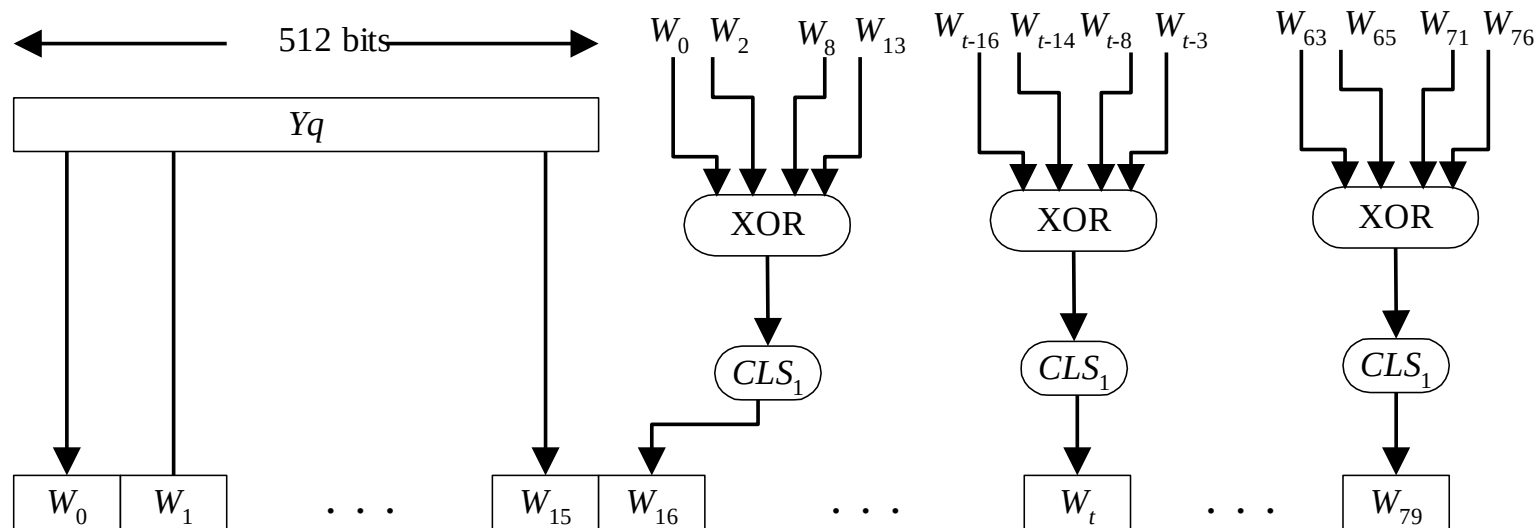


图6-11 SHA分组处理所需的80个字的产生过程

- 消息扩展（将消息从16个32位扩展为80个32位）
- $W_t = M_t$, for $t=0$ to 15
- $W_t = (W_{t-3} \text{ xor } W_{t-8} \text{ xor } W_{t-14} \text{ xor } W_{t-16}) \lll 1$, for $t=16$ to 79

2024



3、SHA-1的压缩函数:

- 步骤(3)到步骤(5)的处理过程可总结如下:

$$\begin{aligned} CV_0 &= IV \\ CV_{q+1} &= SUM_{32}(CV_q, ABCDE_q) \\ MD &= CV_L \end{aligned}$$

- 其中IV是第3步定义的缓冲区ABCDE的初值,
- $ABCDE_q$ 是第q个消息分组经最后一轮处理过程处理后的输出,
- L是消息（包括填充位和长度字段）的分组数,
- **SUM**₃₂ 是对应字的模 2^{32} 加法,
- MD为最终的摘要值

王立华



3、SHA-1的压缩函数： 算法

过程如下：

For $t=0$ to 79

$\text{temp} = (a \lll 5) + f_t(b, c, d) + e + W_t + K_t$

$e = d$

$d = c$

$c = b \lll 30$

$b = a$

$a = \text{TEMP}$

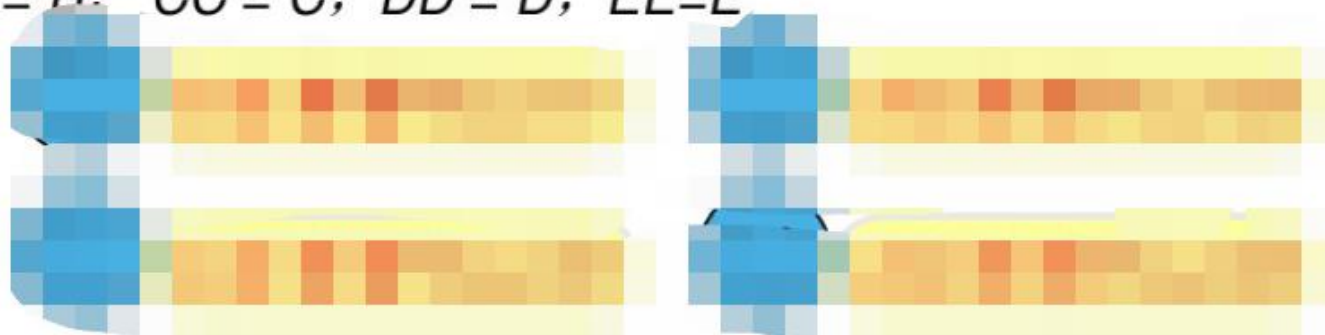
2022/4/24



3、SHA-1的压缩函数： 算法

SHA-1 算法如下

- ①设 A 、 B 、 C 、 D 、 E 是5个32位的寄存器，其初值(用十六进制表示)分别为
 $A=67452301$ 、 $B=efcdab89$ 、 $C=98badcfe$ 、 $D=10325476$ 、 $E=c3d2e1f0$
- ②对 $i = 0$ 至 $N/16 - 1$ 执行第3步至第10步
- ③对 $j = 0$ 至 15 执行 $X[j] = M[16i + j]$
- ④ 对 $j = 16$ 至 79 执行
 $X[j] = (X[j - 3] \oplus X[j - 8] \oplus X[j - 14] \oplus X[j - 16]) \ll 1$
- ⑤将寄存器 A 、 B 、 C 、 D 、 E 中的值存储到另外四个寄存器 AA 、 BB 、 CC 、 DD 中， $AA = A$ ， $BB = B$ ， $CC = C$ ， $DD = D$ ， $EE = E$
- ⑥ 执行Round1
- ⑦ 执行Round2
- ⑧ 执行Round3
- ⑨ 执行Round4
- ⑩ $A = A + AA$ ， $B = B + BB$ ， $C = C + CC$ ， $D = D + DD$ ， $E = E + EE$



2024



4、SHA-1与MD5的比较

- 由于SHA-1与MD5都是由MD4演化而来，所以两个算法极为相似：
 - ① 抗穷搜索攻击的强度：由于SHA和MD5的消息摘要长度分别为160和128，所以用穷搜索攻击寻找具有给定消息摘要的消息分别需做 $O(2^{160})$ 和 $O(2^{64})$ 次运算，而用穷搜索攻击找出具有相同消息摘要的两个不同消息分别需做 $O(2^{80})$ 和 $O(2^{64})$ 次运算。因此**SHA抗击穷搜索攻击的强度高于MD5抗击穷搜索攻击的强度。**
 - ② 抗击密码分析攻击的强度：由于**SHA-1的设计准则未被公开**，所以它抗击密码分析攻击的强度较难判断，**似乎高于MD5的强度。**

王华



4、SHA-1与MD5的比较

- 由于SHA与MD5都是由MD4演化而来，所以两个算法极为相似：
 - ③ 速度：由于两个算法的主要运算都是模 2^{32} 加法，因此都易于在32位结构上实现。但比较起来SHA的迭代步数(80步)多于MD5的迭代步数（64步），所用的缓冲区（160比特）大于MD5使用的缓冲区（128比特），因此在相同硬件上实现时，**SHA-1的速度慢于MD5的速度。**
 - ④ 简洁与紧致性：两个算法描述起来都较为简单，实现起来也较为简单，**都不需要大的程序和代换表。**

王卫华



4、SHA-1与MD5的比较

- 由于SHA与MD5都是由MD4演化而来，所以两个算法极为相似：
 - ⑤ 数据的存储方式：**MD5使用小端方式**，**SHA使用大端方式**。两种方式相比看不出哪个更具优势，之所以使用两种不同的存储方式是因为**设计者最初实现各自的算法时，使用的机器的存储方式不同**。

2022/4/24



5、SHA-1的安全性

2004年，Joux找出了SHA-0的碰撞，他的攻击方法需要 2^{51} 次运算。同年Biham等找出了40步的SHA-1的碰撞。2005年，山东大学王小云等提出了对SHA-1的碰撞搜索攻击，该方法用于攻击完全版的SHA-0时，所需的运算次数少于 2^{39} ；攻击58步的SHA-1时，所需的运算次数少于 2^{33} 。他们还分析指出，用他们的方法攻击70步的SHA-1时，所需的运算次数少于 2^{50} ；而攻击80步的SHA-1时，所需的运算次数少于 2^{69} 。

王立华



6、SHA-1总结

- SHA-1产生160比特的输出。
- SHA=MD4+扩展转换+附加轮+更好的**雪崩效应**。
- 目前为至SHA是安全的，即没有找到有效的攻击方法。

王卫华



7、SHA-2

- SHA-2，名称来自于安全散列算法2（英语：Secure Hash Algorithm 2）的缩写，一种密码散列函数算法标准，由美国国家安全局研发，由美国国家标准与技术研究院（NIST）在2001年发布。
- SHA-2属于SHA算法之一，是SHA-1的后继者。
- SHA-2又可再分为六个不同的算法标准，包括了：SHA-224、SHA-256、SHA-384、SHA-512、SHA-512/224、SHA-512/256。

王华



8、SHA-3

- SHA-3第三代安全散列算法 (Secure Hash Algorithm 3)，之前名为Keccak（念作/^lkɛtʃæk/或/kɛtʃɑ:k/）算法。
- Keccak 是一个**加密散列算法**，由Guido Bertoni，Joan Daemen，Michaël Peeters，以及Gilles Van Assche在RadioGatún上设计。
- 2012年10月2日，Keccak 被选为NIST散列函数竞赛的胜利者。
- SHA-3**并不是要取代SHA-2**，因为SHA-2并没有出现明显的弱点。由于对MD5、SHA-0和SHA-1出现成功的破解，NIST感觉需要一个与之前算法不同的，可替换的加密散列算法，也就是SHA-3。
- 2014年，NIST发布了FIPS202 的草案 “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions”。
- 2015年8月5日，FIPS 202 最终被 NIST 批准。

2022/4/24